

Валидация входных данных и обработка ошибок

Защищаем REST API от некорректных данных и формируем предсказуемые ответы

Результаты занятия

- ✓ Объяснять необходимость валидации входных данных
- ✓ Проверять обязательные поля, типы и ограничения значений
- ✓ Создавать middleware для валидации req.body
- ✓ Различать ошибки клиента и ошибки сервера
- ✓ Передавать ошибки через next(error)
- ✓ Создавать централизованный error-handling middleware
- ✓ Возвращать единообразные JSON-ответы об ошибках

Учебные вопросы

- 1 Зачем нужна валидация** Почему нельзя доверять данным, полученным от клиента
- 2 Правила проверки данных** Обязательные поля, типы, диапазоны и допустимые значения
- 3 Валидация через middleware** Как остановить некорректный запрос до выполнения controller
- 4 Ошибки в Express** `next(error)`, ожидаемые ошибки и ошибки сервера
- 5 Централизованная обработка ошибок** Единый error handler для всего REST API



Учебный вопрос 1. Зачем нужна валидация

Любые данные клиента необходимо считать потенциально некорректными

Откуда сервер получает входные данные

- `req.params` содержит значения из параметров маршрута.
- `req.query` содержит параметры строки запроса.
- `req.body` содержит данные тела HTTP-запроса.
- Значения формируются на стороне клиента и могут быть ошибочными.
- Клиент может отправить отсутствующее поле, неправильный тип или недопустимое значение.

Главная идея

Получить данные из HTTP-запроса недостаточно — перед использованием их необходимо проверить.

Некорректный товар

Запрос: POST /api/products

```
{  
  "name": "",  
  "price": -500,  
  "category": 15,  
  "inStock": "yes"  
}
```

Пояснение

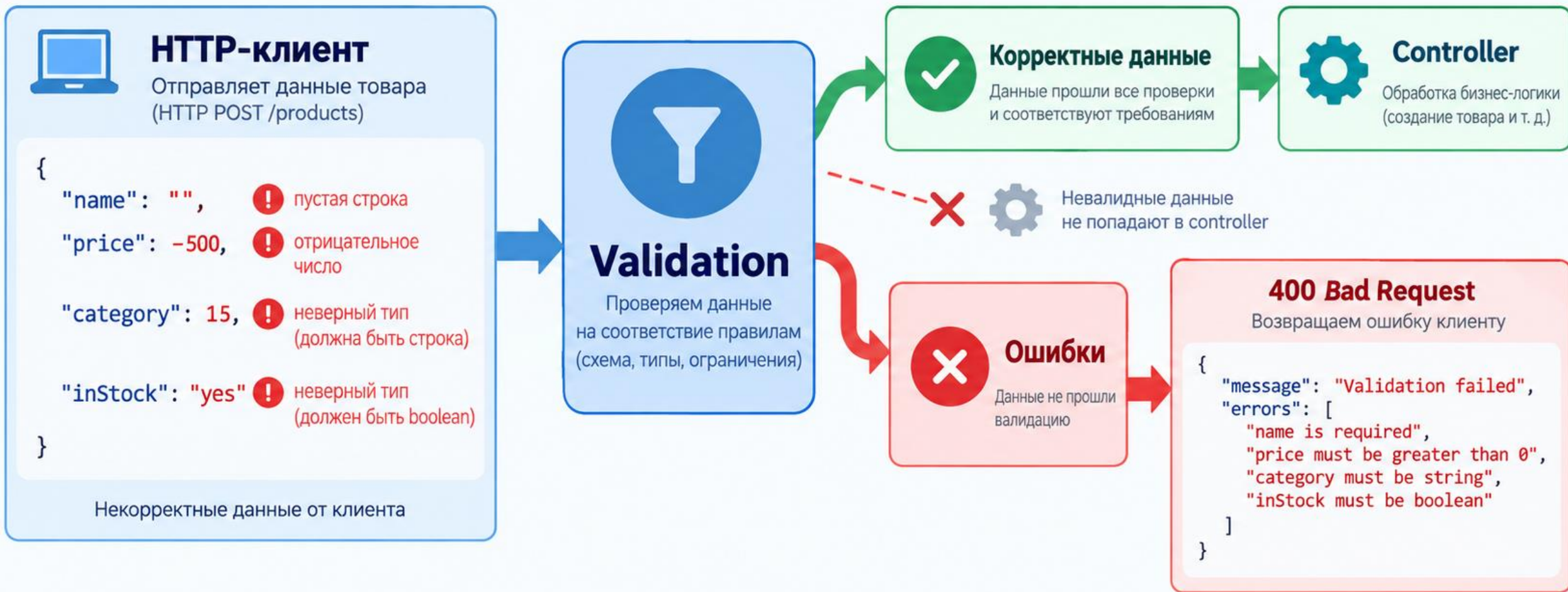
- JSON синтаксически корректен, но данные не соответствуют требованиям нашего Product API.

Что произойдёт без валидации

- В каталог может попасть товар без названия.
- Цена может оказаться отрицательной или строкой.
- Логическое поле inStock может получить произвольное значение.
- Ошибочные данные распространяются на последующие операции приложения.
- Frontend получает непредсказуемые результаты.
- После подключения базы данных некорректные данные могут быть сохранены постоянно.

Данные клиента до и после валидации

Валидация защищает приложение от некорректных данных и делает его более надёжным



Данные клиента нельзя передавать в бизнес-логику без проверки.

Всегда валидируйте входящие данные!

Разбор JSON и валидация — разные задачи

`express.json()`

- Разбирает JSON-тело HTTP-запроса
- Формирует `req.body`
- Позволяет JavaScript работать с объектом
- Не знает правил нашего товара
- Не проверяет бизнес-требования к полям

Валидация

- Проверяет структуру полученных данных
- Проверяет обязательные поля
- Проверяет типы значений
- Проверяет допустимые диапазоны
- Решает, можно ли продолжать обработку

Что можно валидировать

- Наличие обязательного поля.
- Тип значения: `string`, `number`, `boolean`.
- Пустые строки.
- Минимальное и максимальное значение числа.
- Допустимые варианты значения.
- Формат строки.
- Связь нескольких полей между собой.



Учебный вопрос 2. Правила проверки данных

Определяем требования к объекту Product

Правила валидации Product

Поле	Тип	Правило
name	string	Обязательное, непустая строка
price	number	Обязательное, больше 0
category	string	Обязательное, непустая строка
inStock	boolean	Обязательное логическое значение

Проверка обязательных полей

- Для создания товара требуется полный набор необходимых данных.
- Отсутствие поля отличается от неправильного значения.
- Пустая строка name также не является корректным названием.
- Проверка должна выполняться до вызова бизнес-логики.
- При ошибке клиент должен получить понятное описание проблемы.

Ответ при ошибке валидации

Запрос: POST /api/products с price = -500

```
{
  "message": "Validation
failed",
  "errors": [
    "price must be greater than
0"
  ]
}
```

Пояснение

- API отклоняет запрос до создания товара и возвращает клиенту причину ошибки.

Уровни проверки входных данных

На примере поля «price» (создание товара)

Безопасные приложения начинают с проверки данных

1

Поле существует

Поле «price» есть в запросе



```
{  
  "price": 100  
}
```



2

Правильный тип

Значение «price» имеет тип number



```
"price": 100  
(тип number)
```



3

Допустимое значение

Значение «price» больше 0



```
"price": 100  
(100 > 0)
```



4

Бизнес-ограничение

Значение «price» допустимо для создания товара



```
"price": 100  
(соответствует правилам приложения)
```



Проверка пройдена!

Запрос можно передать дальше (создать товар).



Ошибка:
нет поля «price»

Обработка остановлена



Ошибка:
строка вместо числа
"price": "100"

Обработка остановлена



Ошибка:
price <= 0
"price": 0
(или отрицательное число)

Обработка остановлена



Ошибка:
нарушено правило приложения
например, цена слишком большая или не разрешена для данного типа товара

Обработка остановлена



Ошибка на любом уровне останавливает обработку запроса.

Проверяй данные — делай систему надежнее!

Функция проверки данных товара

```
export function validateProductData(data) {  
  const errors = [];  
  
  if (typeof data.name !== 'string' || !data.name.trim())  
    errors.push('name is required');  
}  
  
  if (typeof data.price !== 'number' || data.price <= 0)  
    errors.push('price must be greater than 0');  
}  
  
  if (typeof data.category !== 'string' || !data.category.trim())  
    errors.push('category is required');  
}  
  
  if (typeof data.inStock !== 'boolean') {  
    errors.push('inStock must be boolean');  
  }  
  
  return errors;  
}
```

Что важно

- Функция получает обычный JavaScript-объект.
- Все найденные проблемы записываются в массив errors.
- Проверяются типы и допустимые значения.
- Пустой массив означает, что проверка пройдена.
- Функция не работает с req и res.

Результат функции validateProductData

```
const errors = validateProductData({  
  name: '',  
  price: -100,  
  category: 'Components',  
  inStock: 'yes'  
});
```

Результат

```
[  
  "name is required",  
  "price must be greater than 0",  
  "inStock must be boolean"  
]
```

Пояснение

- Одна проверка может обнаружить сразу несколько проблем и сообщить клиенту обо всех найденных ошибках.

Валидация POST и PATCH

POST /api/products

- Создаётся новый ресурс
- Проверяем наличие обязательных полей
- Проверяем все необходимые типы
- name, price, category и inStock должны присутствовать
- Некорректный объект не создаётся

PATCH /api/products/:id

- Изменяется существующий ресурс
- Разрешено передать только часть полей
- Проверяем только реально переданные свойства
- Переданные значения должны иметь правильный тип
- Неизменяемые свойства можно не отправлять



Учебный вопрос 3. Валидация через middleware

Проверяем запрос до передачи управления controller

Почему валидацию удобно выполнять в middleware

- Middleware выполняется до controller.
- Одинаковую проверку можно подключать к нескольким маршрутам.
- Некорректный запрос можно остановить раньше.
- Controller получает уже проверенные входные данные.
- Код controller остаётся компактным.
- Route наглядно показывает, какие проверки выполняются перед обработчиком.

Middleware validateCreateProduct

```
export function validateCreateProduct(req, res, next) {  
  const errors = validateProductData(req.body);  
  
  if (errors.length > 0) {  
    return res.status(400).json({  
      message: 'Validation failed',  
      errors  
    });  
  }  
  
  next();  
}
```

Что важно

- Middleware получает данные из req.body.
- validateProductData возвращает найденные ошибки.
- Если ошибки есть, цепочка завершается ответом 400.
- Если данные корректны, вызывается next().
- После next() управление переходит controller.

Подключаем validation middleware к маршруту

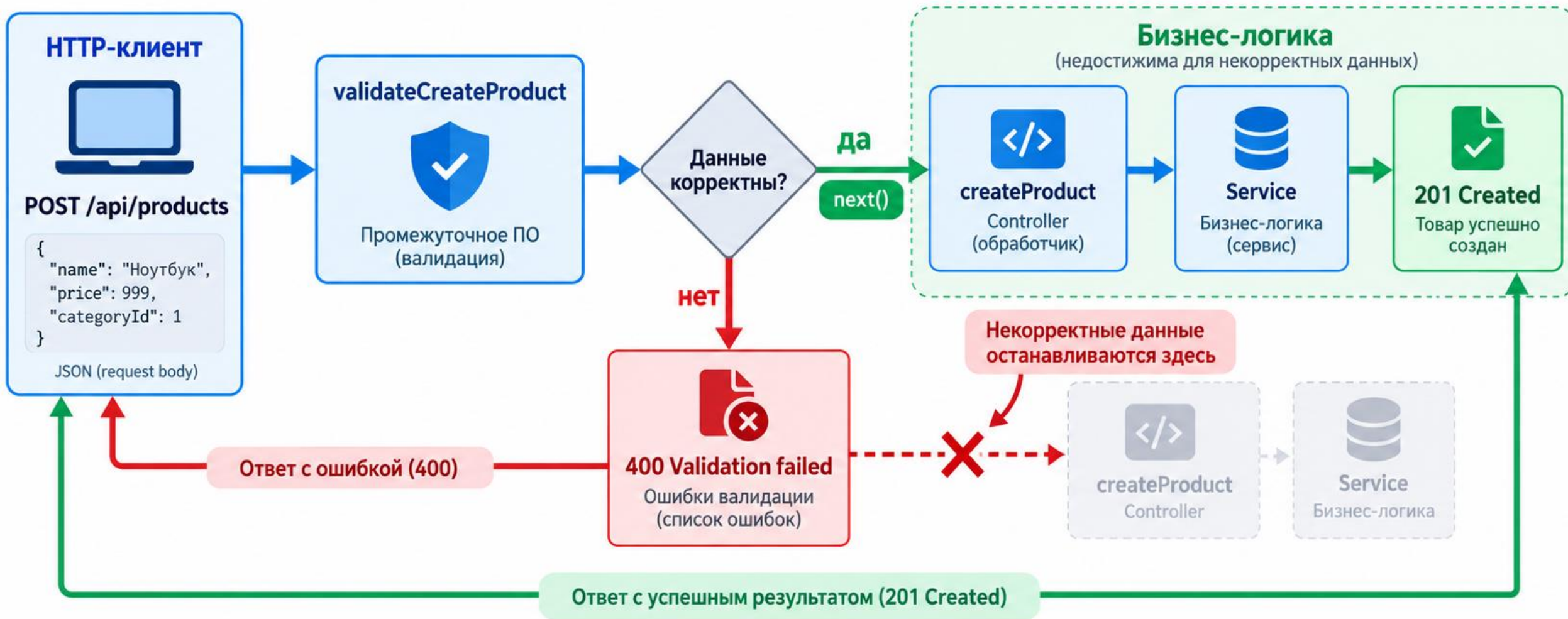
```
router.post(
  '/',
  validateCreateProduct,
  createProduct
);
```

Что важно

- Сначала Express вызывает validateCreateProduct.
- Controller createProduct запускается только после next().
- Некорректные данные не доходят до controller.
- Порядок функций в маршруте имеет значение.

Валидация как фильтр перед Controller

Пример обработки POST-запроса в Express.js



Некорректные данные не достигают controller и service.

Валидация PATCH-запроса

```
export function validateUpdateProduct(req, res, next) {
  const { name, price, category, inStock } = req.body;
  const errors = [];

  if (name !== undefined &&
      (typeof name !== 'string' || !name.trim())) {
    errors.push('name must be non-empty string');
  }

  if (price !== undefined &&
      (typeof price !== 'number' || price <= 0)) {
    errors.push('price must be greater than 0');
  }

  if (category !== undefined &&
      (typeof category !== 'string' || !category.trim())) {
    errors.push('category must be non-empty string');
  }

  if (inStock !== undefined && typeof inStock !== 'boolean') {
    errors.push('inStock must be boolean');
  }

  if (errors.length > 0) {
    return res.status(400).json({ message: 'Validation failed', errors });
  }

  next();
}
```

Что важно

- PATCH не требует наличия всех полей.
- Каждое свойство проверяется только тогда, когда оно передано.
- Некорректное значение приводит к ответу 400.
- Корректный запрос передаётся controller через next().

Валидация и нормализация данных

- Валидация отвечает на вопрос: допустимы ли данные.
- Нормализация приводит допустимые данные к единому виду.
- Например, строку `name` можно очистить методом `trim()`.
- Не следует незаметно исправлять значение, если это меняет его смысл.
- После проверки `controller` и `service` должны получать предсказуемые данные.

Валидация и нормализация

Валидация

- Проверяет корректность
- Ищет отсутствующие поля
- Проверяет типы
- Проверяет ограничения
- Может отклонить запрос

Нормализация

- Приводит данные к единому виду
- Удаляет лишние пробелы
- Подготавливает значение к использованию
- Не должна скрывать серьёзные ошибки
- Выполняется только для допустимых данных

Где выполняется валидация в нашей архитектуре

- Route определяет, какой validation middleware нужно вызвать.
- Validation middleware работает с входными HTTP-данными.
- Controller получает проверенные параметры.
- Service выполняет бизнес-операцию с обычными JavaScript-значениями.
- Service не должен зависеть от req и res.
- Ошибочный HTTP-запрос желательно остановить до вызова service.

Главная идея

**Request → Validation
Middleware → Controller →
Service**



Учебный вопрос 4. Ошибки в Express

Не все ошибки возникают из-за неправильных входных данных

Ожидаемые и неожиданные ошибки

Ожидаемая ошибка

- Некорректные входные данные
- Товар с указанным id не найден
- Клиент может получить понятное сообщение
- Обычно соответствует статусам 4xx
- Является частью нормальной работы API

Неожиданная ошибка

- Ошибка программного кода
- Сбой внешнего сервиса
- Ошибка обращения к базе данных
- Не должна раскрывать внутренние детали клиенту
- Обычно приводит к статусу 500

Коды ошибок, которые используем

Код	Ситуация	Пример
400	Некорректный запрос	price = -500
404	Ресурс не найден	Нет товара с id 999
404	Маршрут не существует	GET /api/unknown
500	Внутренняя ошибка сервера	Неожиданное исключение

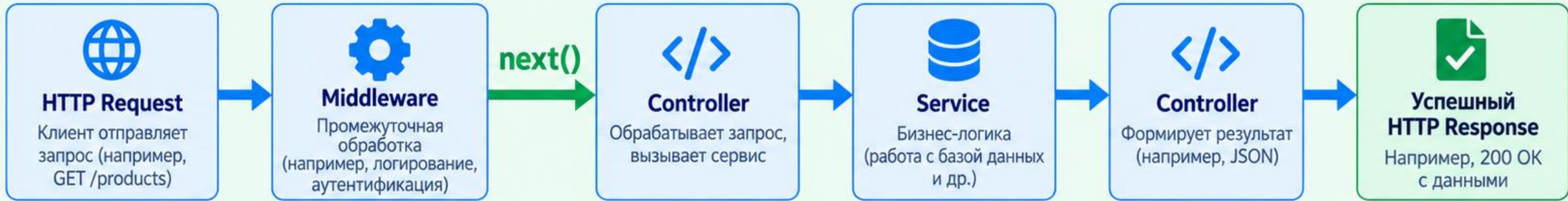
Обычный путь запроса и путь ошибки

Разница между `next()` и `next(error)` в Express.js



Обычный путь запроса (`next()`)

Запрос последовательно проходит все обработчики и возвращает успешный ответ.



Путь ошибки (`next(error)`)

Ошибка может возникнуть в любом месте. Она передается в middleware обработки ошибок и пропускает обычную цепочку.



`next()` продолжает обычную цепочку, а `next(error)` переводит запрос в централизованную обработку ошибок.

Что делает `next(error)`

- Обычный `next()` передаёт управление следующему `middleware`.
- `next(error)` сообщает Express о возникшей ошибке.
- Обычные `middleware` и обработчики после этого пропускаются.
- Express ищет `middleware` обработки ошибок.
- Так ошибку можно передать в единый централизованный обработчик.

Главная идея

`next()` продолжает обычную цепочку, а `next(error)` переключает запрос на цепочку обработки ошибок.

Собственный класс AppError

```
export class AppError extends Error {  
  constructor(message, statusCode = 500) {  
    super(message);  
    this.statusCode = statusCode;  
  }  
}
```

Что важно

- AppError расширяет стандартный класс Error.
- message содержит описание ошибки.
- statusCode хранит HTTP-код, который должен получить клиент.
- Значение 500 используется по умолчанию.

Controller передаёт ошибку 404

```
export function getProductById(req, res, next) {
  const id = Number(req.params.id);
  const product = findProductById(id);

  if (!product) {
    return next(
      new AppError('Product not found', 404)
    );
  }

  res.json(product);
}
```

Что важно

- Service возвращает результат поиска или undefined.
- Controller решает, что отсутствие товара соответствует 404.
- Вместо самостоятельного JSON-ответа controller создаёт AppError.
- next(error) передаёт ошибку централизованному обработчику.

Зачем передавать ошибки в единый обработчик

- Controllers не повторяют одинаковый код формирования ошибок.
- Все ошибки получают единый формат JSON.
- Проще изменить формат ответа сразу для всего API.
- В одном месте можно выполнять журналирование ошибок.
- Неожиданные ошибки можно скрыть от клиента и вернуть безопасное сообщение.
- Код приложения становится проще сопровождать.

Ошибка внутри controller

```
export function demoController(req, res, next) {  
  try {  
    throw new Error('Unexpected error');  
  } catch (error) {  
    next(error);  
  }  
}
```

Что важно

- Исключение сохраняется в объекте error.
- Ошибка передаётся Express через next(error).
- Controller не формирует окончательный ответ.
- Ответ будет сформирован error-handling middleware.



Учебный вопрос 5. Централизованная обработка ошибок

Один error handler формирует ошибки всего REST API

Error-handling middleware

- Error-handling middleware — специальный middleware Express с четырьмя параметрами: `err`, `req`, `res`, `next`.
- Получает ошибки, переданные через `next(error)`.
- Обычно размещается после маршрутов приложения.
- Определяет HTTP-код ответа.
- Формирует единый JSON-ответ.
- Может журналировать информацию об ошибке.

Централизованный поток обработки ошибки

Все ошибки из разных частей приложения попадают в один обработчик, который формирует единый и понятный JSON-ответ.

Источники ошибок



Product Controller

```
const product = await
  findProductById(id);
if (!product) {
  throw new NotFoundError('Product
    not found');
```

Например: товар не найден



Category Controller

```
if (!name) {
  throw new ValidationError(
    'Category name is required'
  );
}
```

Например: неверные данные



Middleware

(валидация, аутентификация и др.)

```
if (!req.user) {
  return next(new UnauthorizedError(
    'Authentication required'
  ));
}
```

Например: нет авторизации



Неизвестный маршрут (404)

```
app.use('*', (req, res, next) => {
  next(new NotFoundError(
    'Route not found'
  ));
});
```

Например: маршрут не существует

Любая
ошибка



Единый обработчик ошибок

errorHandler(err, req, res, next)

```
app.use((err, req, res, next) => {
  console.error(err);

  let status = err.statusCode || 500;
  let message = err.message ||
    'Internal Server Error';

  res.status(status).json({
    success: false,
    message,
    ...(process.env.NODE_ENV === 'development'
      ? { stack: err.stack }
      : {}),
  });
});
```

Ответственность:

- Получить любую ошибку из приложения
- Определить корректный HTTP-статус
- Сформировать единый формат JSON-ответа
- Скрыть технические детали в production
- Отправить ответ клиенту

Единый
JSON-ответ



Клиент

Получает понятный и единообразный ответ

```
{
  "success": false,
  "message": "Product not found",
  "status": 404
}
```

Один формат ответа для всех ошибок,
независимо от источника.

Типичные статусы ошибок

400 Bad Request

- Неверные данные
- Ошибки валидации
- Отсутствуют обязательные поля

404 Not Found

- Ресурс не найден
- Неверный маршрут
- Несуществующий ID

500 Internal Server Error

- Неожиданные ошибки
- Сбои в базе данных
- Внешние сервисы недоступны
- Программные ошибки



Преимущество: централизованная обработка упрощает поддержку, обеспечивает единый формат ответов и улучшает безопасность приложения.

Централизованный errorHandler

```
export function errorHandler(err, req, res, next) {  
  const statusCode = err.statusCode || 500;  
  
  console.error(err);  
  
  res.status(statusCode).json({  
    message:  
      statusCode === 500  
        ? 'Internal server error'  
        : err.message  
  });  
}
```

Что важно

- Error handler обязательно принимает четыре аргумента.
- Для AppError используется указанный statusCode.
- Для неизвестной ошибки используется 500.
- Внутренняя ошибка записывается в консоль сервера.
- Клиент не получает технические подробности неожиданной ошибки.

Middleware для неизвестного маршрута

```
export function notFound(req, res, next) {  
  next(  
    new AppError(  
      `Route ${req.method} ${req.originalUrl} not found`,  
      404  
    )  
  );  
}
```

Что важно

- Middleware размещается после всех известных routes.
- Если ни один маршрут не ответил, запрос попадает сюда.
- Создаётся ошибка со статусом 404.
- Ошибка передаётся общему errorHandler.

Правильный порядок middleware в app.js

```
app.use(express.json());  
  
app.use('/api/products', productRoutes);  
app.use('/api/categories', categoryRoutes);  
  
app.use(notFound);  
app.use(errorHandler);
```

Что важно

- Сначала выполняется разбор JSON.
- Затем подключаются маршруты приложения.
- После routes находится обработчик неизвестного маршрута.
- Последним подключается errorHandler.
- Порядок регистрации определяет порядок прохождения запроса.

Единый формат ошибки

Запрос: GET /api/products/999

```
{  
  "message": "Product not found"  
}
```

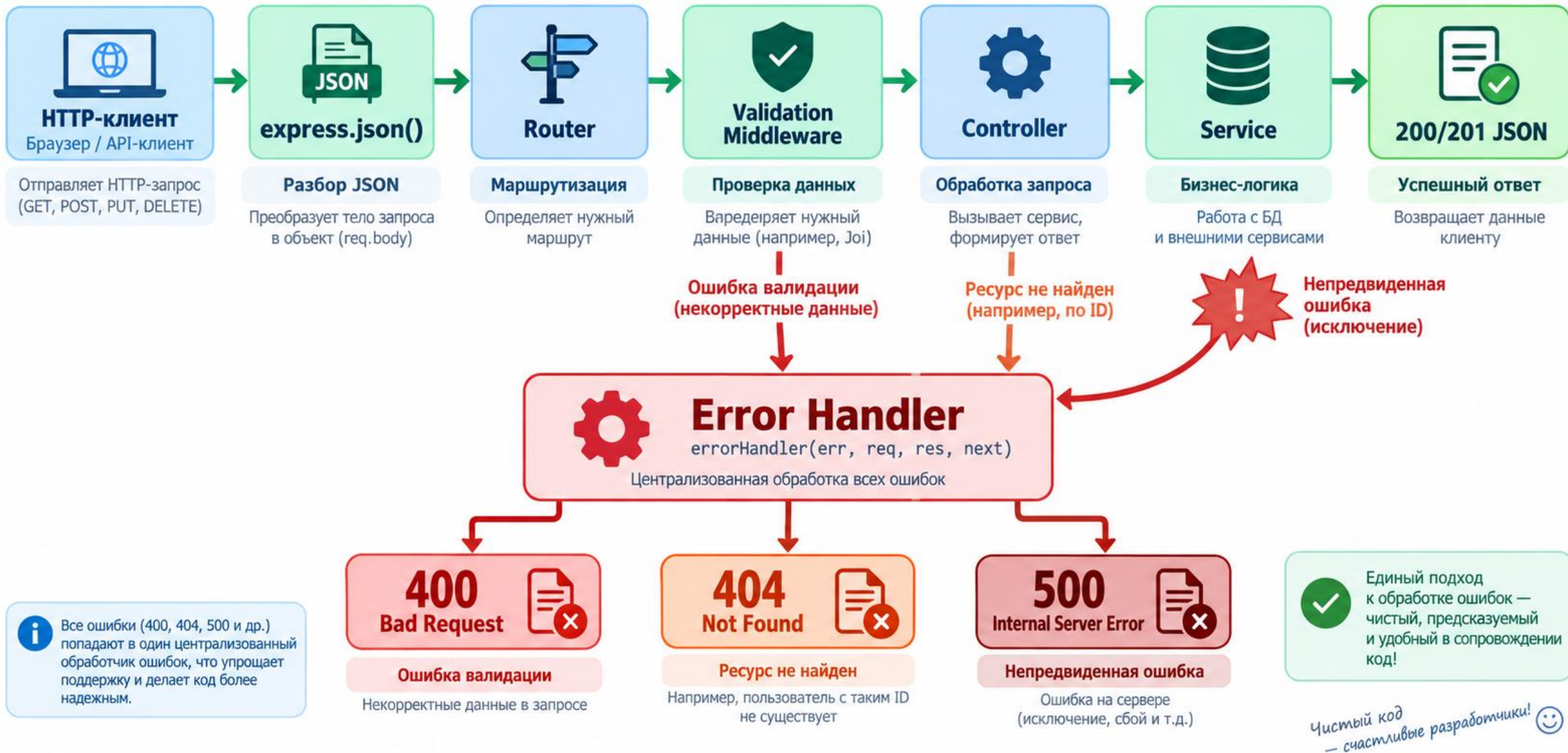
Пояснение

- Клиент получает предсказуемую структуру ответа независимо от того, в каком controller возникла ошибка.

Полный путь запроса с валидацией и обработкой ошибок

Слайд №44

Запрос выполнен успешно!



Что происходит при POST /api/products

- 1 Клиент отправляет запрос** POST /api/products содержит JSON с данными нового товара.
- 2 `express.json()` разбирает тело** JSON преобразуется в объект `req.body`.
- 3 `validateCreateProduct` проверяет данные** Некорректные данные приводят к ответу 400 Validation failed.
- 4 **Controller** получает корректные данные** Если `validation middleware` вызвал `next()`, запрос передаётся `createProduct`.
- 5 **Service** выполняет операцию** `addProduct` создаёт новый товар и возвращает созданный объект.
- 6 **Controller** отправляет ответ** При успешном создании клиент получает 201 Created и JSON нового товара. Неожиданная ошибка передаётся `errorHandler`.

Что нельзя отправлять клиенту при внутренней ошибке

- Полный stack trace приложения.
- Физические пути к файлам сервера.
- Тексты SQL-запросов с внутренними деталями.
- Пароли, токены и секретные ключи.
- Другую техническую информацию, которая не нужна клиенту.
- Для клиента достаточно безопасного сообщения и подходящего HTTP-кода.

Мини-практика

Добавьте в Product API проверку входных данных и централизованную обработку ошибок.

Задание

- Создать `validateCreateProduct`
- Проверить `name`, `price`, `category` и `inStock`
- Подключить `validation middleware` перед `createProduct`
- Создать класс `AppError`
- Передавать ошибку 404 через `next(error)`
- Создать `notFound middleware`
- Создать общий `errorHandler`
- Проверить ответы 400, 404 и 500

Критерии проверки

- Некорректный POST не создаёт товар
- Ошибка валидации возвращает 400
- Отсутствующий товар возвращает 404
- Неизвестный маршрут возвращает 404
- Неожиданная ошибка возвращает 500
- Все ошибки имеют предсказуемый JSON-формат

Контрольные вопросы

1. Почему нельзя доверять данным из `req.body`?
2. Чем разбор JSON отличается от валидации?
3. Какие правила проверяются для объекта `Product`?
4. Почему `POST` и `PATCH` требуют разных правил валидации?
5. Зачем выполнять валидацию в `middleware`?
6. Чем `next()` отличается от `next(error)`?
7. Что означает HTTP 400?
8. Когда REST API должен вернуть 404?
9. Почему `error-handling middleware` принимает четыре аргумента?
10. Почему внутренние детали ошибки не следует отправлять клиенту?

Итоги занятия

1

Входные данные клиента необходимо проверять до выполнения бизнес-логики.

2

Валидация проверяет обязательные поля, типы и ограничения значений.

3

Validation middleware позволяет остановить некорректный запрос до controller.

4

Ошибки клиента обычно относятся к классу 4xx, внутренние ошибки сервера — к 5xx.

5

next(error) передаёт ошибку специальному error-handling middleware.

6

Централизованный errorHandler создаёт единый формат ошибок для всего API.

7

Клиенту не следует раскрывать внутренние технические детали неожиданной ошибки.